PROJECT REPORT
ON

"DETECTION OF CIPHER TYPES USING ARTIFICIAL NEURAL NETWORKS"

SUBMITTED IN PARTIAL FULFILMENT FOR THE
DEGREE OF BACHELORS OF TECHNOLOGY

SUBMITTED BY

Rajdeep Ghosh (510519002)
Swarnali Mondal (510510103)
Krishnendu Bera (510519004)
Arnab Sen (510519006)
Sanjana Chakravarty (510519038)
Abhiroop Mukherjee (510519109)

UNDER THE GUIDANCE OF

DR. MALAY KULE



DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY

INDIAN INSTITUTE OF ENGINEERING SCIENCE AND TECHNOLOGY, SHIBPUR
Howrah, West Bengal, India – 711103

(Academic Year: 2022–2023)

INDIAN INSTITUTE OF ENGINEERING SCIENCE AND TECHNOLOGY, SHIBPUR
Howrah, West Bengal, India – 711103

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY



Certificate

It is certified hereby that this report, titled "Detection of Cipher Types using Artificial Neural Networks", and all the attached documents herewith are authentic records of Rajdeep Ghosh (510519002), Swarnali Mondal (510510103), Krishnendu Bera (510519004), Arnab Sen (510519006), Sanjana Chakravarty (510519038), and Abhiroop Mukherjee (510519109) from the Prestigious Department of Computer Science And Technology of the Distinguished and Respected IIEST Shibpur under my guidance.

The works of these students satisfy all the requirements for which it is submitted. To the extent of my knowledge, it has not been submitted to any different institutions for the award of a degree.

Dr. Malay Kule
Asst. Professor

Dr. Asit Kumar Das
Head Of Department

ACKNOWLEDGEMENT

We, as the students of IIEST, consider ourselves honored to be working with Dr. Malay Kule and Mr. Subinoy Sikdar. The success of this project would not have been possible without his useful insights, appropriate guidance, and necessary criticism.

We would pass our token of gratitude to the Department of Computer Science And Technology as well for providing us with the opportunity to be able to tackle real-world problems while improving our problem-solving ability and thinking capacity by organizing this project. We all have learned quite a handful of new skills and are eager to use them henceforth as well.

Rajdeep Ghosh (510519002)
Swarnali Mondal (510510103)
Krishnendu Bera (510519004)
Arnab Sen (510519006)
Sanjana Chakravarty (510519038)
Abhiroop Mukherjee (510519109)

Contents

1 INTRODUCTION **1**

2 PRELIMINARIES **1**

3 LITERATURE SURVEY **4**

4 PROPOSED WORK **5**
4.1 Dataset Generation 5
4.2 Model using CNN 5
4.3 Model using Transformer 6

5 EXPERIMENTAL RESULTS **9**
5.1 Model using CNN 9
5.2 Model using Transformer 10

6 FUTURE SCOPE **12**

7 CONCLUSION **12**

1 INTRODUCTION

Information is the power in the present day. Information and data should be protected from unauthorized access, distortion, or any unauthorized activity. Cryptography is the study of secret writing. Information is no longer hidden due to the existence of network adversaries. Our communication channel is completely unstable; a third party might covertly eavesdrop and obtain information from the network or from the channel. Here, cryptography is crucial in preventing unauthorized access to our data. Using key(s), cryptography converts plaintext messages into unreadable ciphertext and vice versa. Encryption is the process of employing a key(s) to transform the original message into an unintelligible format, while decryption is the act of using a key(s) to convert the cipher text back to plaintext. The messages can only be encrypted or decrypted by the authorized user.

The project's objective is to examine encrypted text files to find and categorize their cipher types using artificial neural networks. The cipher algorithm classification model is based on the steps:

1. Creation of a collection of cipher texts with various cipher texts.
2. Building a Deep Neural Network (DNN) and training it with a variety of cipher texts and training parameters to enable accurate learning and cipher text classification.
3. Using the trained model to determine the algorithm that might have been used to generate the cipher text.

Our report is organized into seven different sections. This section serves as an introduction, followed by Section 2 that details the preliminaries for this project. Section 3 gives an overview of the papers and tutorials we studied prior to this project. Section 4 elaborates our proposed work with dataset generation and discussion of the models. Section 5 then presents the experimental findings. The next work that we intend to address in the upcoming semester is contained in Section 6, followed by the conclusion in Section 7.

2 PRELIMINARIES

To ramp up for this project, we needed a basic knowledge of neural networks and deep learning and an overview of the classical encryption algorithms that we would be targeting in this project. Having specific knowledge of the encryption algorithms will be useful to particularly train the model.

The ML topics that are covered in this project are:

- **Artificial Neural Network (ANN)** – An artificial neural network (ANN) is a type of machine learning model that is inspired by the structure and function of the human brain. ANNs consist of layers of interconnected neurons, which process input data and generate output values. ANNs can be trained to perform a variety of tasks, such as classification, regression, and clustering.

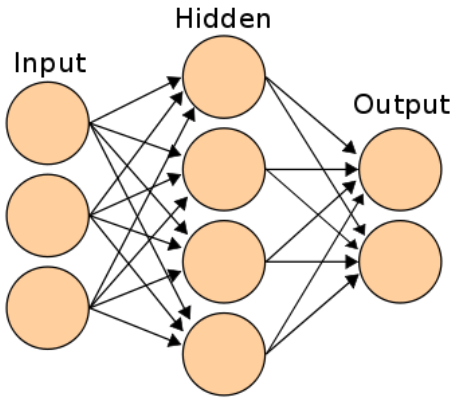


Figure 1: Artificial Neural Network

- **Perceptron** – A perceptron is a type of artificial neural network (ANN) that can be used for binary classification tasks. It consists of a single layer of neurons, with each neuron receiving input from multiple sources, and producing a single output value. The output of a perceptron is determined by a set of weights that are applied to the input values, and a threshold value that is used to determine whether the output is 0 or 1.

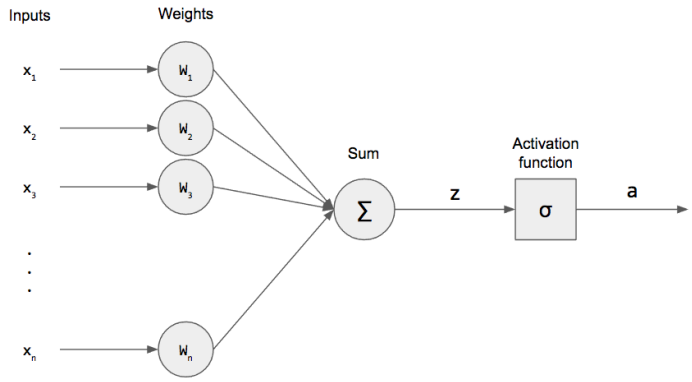


Figure 2: Single Perceptron

- **Recurrent Neural Network (RNN)** – A recurrent neural network (RNN) is a type of ANN that is designed to process sequential data, such as time series data or natural language text. RNNs have feedback connections that allow them to use information from previous time steps to process the current input, which enables them to capture dependencies between input elements over time. RNNs are often used for tasks such as language translation and speech recognition.

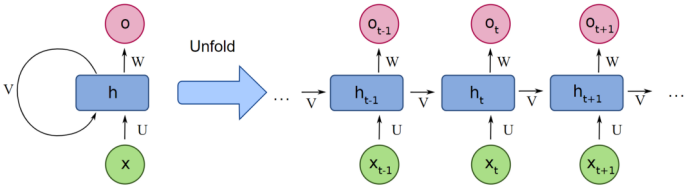


Figure 3: Recurrent Neural Network

- **Convolutional Neural Network (CNN)** – CNNs consist of multiple layers of interconnected neurons, which process the input data in a hierarchical manner. The first layer of a CNN typically consists of a set of filters, which are applied to the input data to extract features such as edges and textures. These features are then passed to the next layer of the CNN, where they are combined and processed to extract more complex features, such as shapes and objects. The output of a CNN is a set of probabilities that indicate the likelihood that the input data belongs to each of the possible classes.

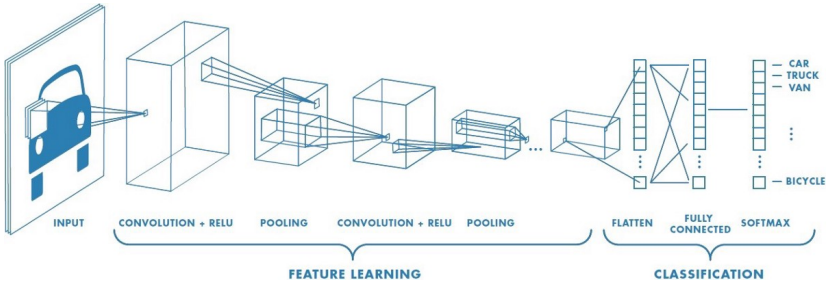


Figure 4: Convolutional Neural Network

- **Transformers** – The "Attention is all you need" paper is a research paper published in 2017 that presents the transformer, a novel neural network architecture for natural language processing (NLP) tasks. The transformer is a type of encoder-decoder model, which is commonly used for tasks such as machine translation and text summarization. The key innovation of the transformer is the use of self-attention, a mechanism that allows the model to learn dependencies between different words in the input sentence. This allows the transformer to process the input sentence in parallel, rather than in a sequential manner, which makes it much more efficient and scalable than previous NLP models.
- **Attention Mechanism** – Self-attention is a mechanism used in the transformer, a neural network architecture for natural language processing (NLP) tasks. Self-attention allows the model to learn dependencies between different words in the input sentence, by computing a weighted sum of the input words based on their relevance to each other.

Below are the ciphers which we used to generate our dataset:

- **Caesar Cipher** – The Caesar cipher is a simple substitution cipher that replaces each letter in a message with a different letter based on a fixed shift value. The shift value determines how many letters to the right or left each letter in the message is replaced with. For example, if the shift value is 3, the letter "A" would be replaced with "D", and the letter "B" would be replaced with "E".

  The Caesar cipher can be represented using the following formula, where the input message is represented by the variable "m" and the shift value is represented by the variable "k":

$$\mathbf{c = (m + k)\ \%\ 26} \tag{1}$$

- **Affine Cipher** – The affine cipher is a type of substitution cipher that uses a combination of linear and modular arithmetic to encrypt and decrypt messages. It uses two keys, a and b, to determine how each letter in the input message is replaced with a different letter in the output message. The affine cipher can be represented using the following formula, where the input message is represented by the variable "m", the key "a" is a number that is coprime to 26, and the key "b" is any number between 0 and 25:

$$\mathbf{c = (a*m + b)\ \%\ 26} \tag{2}$$

- **Vigenère Cipher** – The Vigenère cipher is a polyalphabetic substitution cipher that uses a keyword to encrypt and decrypt messages. The keyword is repeated to create a key that is the same length as the input message, and each letter in the message is replaced with a different letter based on the corresponding letter in the key. The Vigenère cipher is considered to be more secure than the Caesar cipher and the affine cipher, as it uses a different substitution for each letter in the message. The Vigenère cipher can be represented using the following formula, where the input message is represented by the variable "m", the keyword is represented by the variable "k", and the length of the message is represented by the variable "n":

$$\mathbf{c = (m + k)\ \%\ 26} \tag{3}$$

- **Monoalphabetic Substitution Cipher** – The monoalphabetic substitution cipher is a type of substitution cipher that uses a fixed mapping of letters to encrypt and decrypt messages. Each letter in the input message is replaced with a different letter according to the fixed mapping, but the mapping does not change for different letters in the message. This means that the same letter in the input message will always be replaced with the same letter in the output message.

  The monoalphabetic substitution cipher is less secure than the Caesar cipher, the affine cipher, and the Vigenère cipher, as it only uses a single substitution for each letter in the message.

- **Rail Fence Cipher** – The rail fence cipher is a simple transposition cipher that is used to encrypt and decrypt messages by rearranging the letters in the input message. The rail fence cipher is based on the idea of writing the message on a grid of squares, with each letter in the message written in a different square. The message is then read off the grid in a zig-zag pattern, starting from the top left corner and moving to the right and then down.

  The rail fence cipher can be represented using the following formula, where the input message is represented by the variable "m" and the number of rows in the grid is represented by the variable "n":

  ```
  c = m[0] + m[n] + m[2n] + ... + m[(n-1)n] + m[1] + m[n+1] +
  m[2n+1] + ... + m[(n-1)n+1] + ... + m[(n-1)^2]
  ```

  This formula shows how the letters in the input message are rearranged to create the encrypted message

3 LITERATURE SURVEY

Over recent years, artificial neural networks (ANNs) have come to prominence. Applications and research areas for ANNs are numerous and diverse. Their primary objective is the quick filtering, classification, and processing of (mainly) non-linear data, such as speech recognition, picture processing, and language translation. In addition, researchers were able to "train" ANNs to play games (e.g. AlphaStar [1]) or produce artwork in the manner of well-known artists.

On the MysteryTwister C3[2] (MTC3) website in late 2019, Stamp posted a challenge titled "Cipher ID". The task is to identify the type of cipher used to encrypt each of the 500 ciphertexts in the collection, which were encrypted using 5 distinct ciphers and random keys. The individual ciphertexts were then mixed up after each cipher type had been employed exactly 100 times. All prior solvers completed the task by deciphering the ciphertexts using solvers for the 5 various cipher types, despite the author's goal of encouraging others to begin research in the fields of machine learning and cipher type detection. The participants were able to determine the type of encryption the algorithm that was employed after learning the plaintext of each cipher.

We worked on the cipher text detection problem with the help of ANN and in this process, TensorFlow and Pytorch were used. TensorFlow is a free and open-source data flow and math library developed by Google written in Python, C++, and CUDA, and was publicly released in 2015. PyTorch is an open-source machine learning (ML) framework based on the Python programming language and the Torch library which was initially released in the year 2016.

In this section, we present different papers and articles, which we studied before starting with our work. The usage of ANNs in the paper ranges between the emulation of ciphers, the detection of the cipher type, and the recovery of cryptographic keys.

1. Attention Is All You Need[3] (NIPS 2017), here a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train.

2. Detecting the Type of Ciphers Using Artificial Neural Networks[4] (ICHC, 2020), where a first version of an artificial neural network was developed to differentiate between five classical ciphers: simple monoalphabetic substitution, Vigenere, Playfair, Hill, and transposition.

3. In the Language Models are Unsupervised Multitask Learners[5] paper, demonstration of language models that can perform down-stream tasks in a zero-shot setting – without any parameter or architecture modification was done.

4. Language Models are Few-Shot Learners[6], broader societal impacts of this finding and of GPT-3 in general are discussed in this paper

5. Detection of Classical Cipher Types with Feature-Learning Approaches[7], where the applicability of the two feature-learning algorithms, long short-term memory (LSTM) and Transformer, have been explored to overcome the dependency on domain knowledge

6. A systematic evaluation of large language models of code[8] (ICLR 2022), where a new model PolyCoder was released, with 2.7B parameters based on the GPT-2 architecture, that was trained on 249GB of code across 12 programming languages on a single machine.

7. GPT-NeoX-20B: An Open-Source Autoregressive Language Model[9], where GPT-NeoX-20B was introduced which was a 20-billion-parameter autoregressive language model trained on the Pile.

8. Fine-tune a pretrained model[10], a tutorial where a step-by-step process to fine-tune a pre-trained model with a deep learning framework is discussed.

9. Fine-Tuning Transformers for NLP[11], another tutorial on how to fine-tune two different transformer models, BERT and DistilBERT: Sentiment Analysis, and Duplicate Question Detection is covered.

10. How to generate text: using different decoding methods for language generation with Transformers [12], a blog post which gives a brief overview of different decoding strategies and shows how can they be implemented with very little effort using the popular transformers library

4 PROPOSED WORK

4.1 Dataset Generation

The dataset has been generated for five types of ciphers:

- Caesar
- Affine
- Vigenère
- Substitution
- Rail Fence

The plaintext used for encryption was a text from Shakespeare's Hamlet and Macbeth.

For Data Preprocessing, we converted text to lowercase and removed all whitespaces and punctuation.

Each of the classes has 6815 data points, resulting in a total of 34075 data points and each data point consists of the ciphertext (input) and the encryption algorithm (output).

We wrote a Python script for each of the ciphers to encrypt the data into a Pandas DataFrame. The combined DataFrame is shuffled and exported as a CSV file for the resulting dataset.

4.2 Model using CNN

The main intuition behind this method is that CNN uses its kernel as a "local receptive field", which means that a CNN kernel (small in size as we are dealing with stream cipher and not block cipher) can look at its nearby characters and will find some pattern in the ciphertext

So, we first converted every ciphertext into their respective list of tokens using a TextVectorizer, then we converted that list into a 2D text matrix as follows:

$$\begin{bmatrix} a & b & c & d \end{bmatrix} \longrightarrow \begin{bmatrix} a & b & c & d \\ a & b & c & d \\ a & b & c & d \\ a & b & c & d \end{bmatrix}$$

Figure 5: Converting text to 2D text matrix

Note that this conversion depends on the length of the string used, i.e. for a string for length n, we will generate a $n \times n$ matrix". After we generate all variable-sized matrices, we resize them to 60×60 using Bilinear Interpolation.

Now we can see that a kernel (say 3×3 with stride one) can observe its previous as well as its next character and can use that information to find patterns in the ciphertext.

We use a CNN to extract features from the matrix and then we feed those features to a Dense Network to classify between the ciphertext types.

4.3 Model using Transformer

The Transformer in NLP is a novel architecture that aims to solve sequence-to-sequence tasks while handling long-range dependencies with ease. The Transformer was proposed in the paper Attention Is All You Need [3].

Quoting from the paper:
"The Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution."

Here, "transduction" means the conversion of input sequences into output sequences. The idea behind Transformer is to handle the dependencies between input and output with attention and recurrence completely.

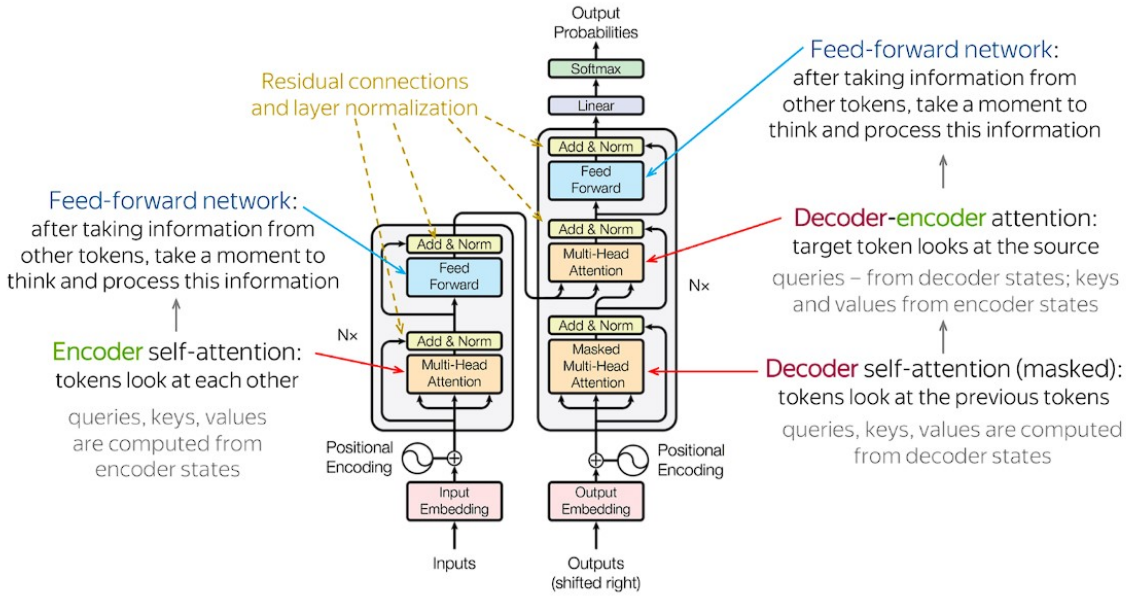


Figure 6: Transformer architecture

The encoder and decoder blocks are actually multiple identical encoders and decoders stacked on top of each other. Both the encoder stack and the decoder stack have the same number of units.

The number of encoder and decoder units is a hyperparameter.

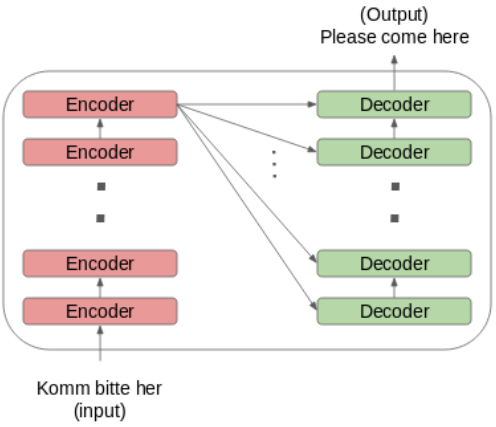


Figure 7: Encoder Decoder Stacks

This is how it works:

- The word embeddings of the input sequence are passed to the first encoder
- These are then transformed and propagated to the next encoder
- The output from the last encoder in the encoder-stack is passed to all the decoders in the decoder-stack as shown in the figure below:

In addition to the self-attention and feed-forward layers, the decoders also have one more layer of Encoder-Decoder Attention layer. This helps the decoder focus on the appropriate parts of the input sequence.

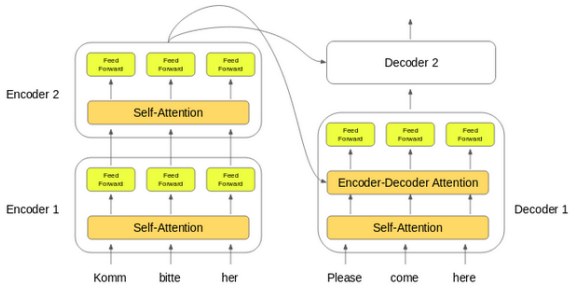


Figure 8: Transformer Working

According to the paper:

"Self-attention, sometimes called intra-attention, is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence."

Consider the following sentence as an input sentence we are feeding into a transformer:

"The animal didn't cross the street because it was too tired"

What does "it" in this sentence refer to? Is it referring to the street or to the animal? It's a simple question to a human, but not as simple to an algorithm.

When the model is processing the word "it", self-attention allows it to associate "it" with "animal".

As the model processes each word (each position in the input sequence), self attention allows it to look at other positions in the input sequence for clues that can help lead to a better encoding for this word.

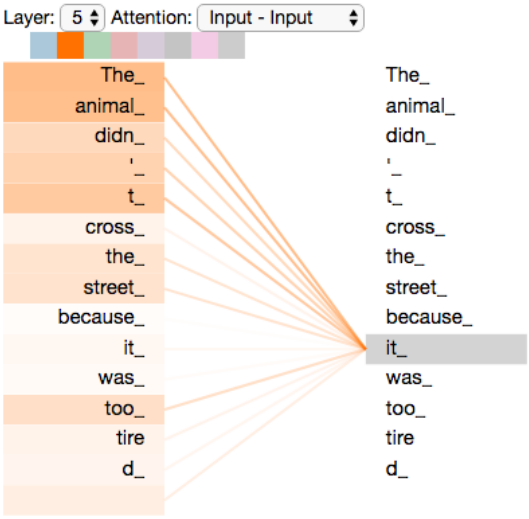


Figure 9: Self Attention

The main intuition behind this method is to compute the relative importance of a ciphertext character with respect to the other ciphertext characters. From our dataset containing encrypted text and the algorithm used for encryption, we create a text prompt from each row for example:

Cipher text: `ymccaygimgamkyyiycuycqocyeagmigkycmyyoyy`
Algorithm: `affine`

The model learns the relative importance of each token and uses it to predict the next. For example, if the model is trained with the above prompt and then we supply the following prompt:

Cipher text: `yycmdkreqluithnalekstndonwothvn`
Algorithm:

The model shall generate the output as "affine" which it learned from training.

In our use case, we have used the GPT-Neo (Generative Pre-Trained) model with 125 million parameters to achieve our objective. It consists of stacks of encoders and decoders put one after the other, of which has been pre-trained on Wikipedia Corpus as well as Common Crawl (This has over 12 Petabytes of data which is 12 years of data uploaded on the internet) datasets for performing extremely well on language-based use cases. Generative, as the word suggests, is for making our code generate text. The architecture is similar to GPT2 except that GPT Neo uses local attention in every other layer with a window size of 256 tokens. All these models are available for free in the Hugging Face transformers library which can be used for a variety of NLP tasks.

The following steps were taken for this model:

1. Each data of the dataset is converted into text prompts that is passed through a tokenizer that creates embedding for each token in our input string.
2. The input dataset has been split into the ratio 80:10:10 for training, testing, and validating purposes.
3. The training and validation set has been used to fine-tune the model's hyperparameters specific to our use-case i.e. detecting the algorithm used in the ciphertext generation.
4. We used the fine-tuned model on our test dataset to check the accuracy of our model.

For our testing purpose, we used various decoding strategies to find the one which gives the best result. Out of the strategies: Greedy Search, Beam Search, and Sampling with set temperature, we found sampling to give the most accurate results of all three. It is a method that is in between the greedy and sampling approach.

The greedy method selects the next token that has the highest probability as the output while sampling selects one of the possible next tokens randomly. When we set a temperature, the randomness is lowered such that the likelihood of high-probability words are increased and that of the low probability are decreased. We found that a **temperature set to 0.7** gave the optimum accuracy on our test data.

5 EXPERIMENTAL RESULTS

5.1 Model using CNN

Our dataset consisted of 34075 rows that were split for training, testing, and validating in the ratio 80:10:10. The following are the sizes for each split:

- Train dataset: 27260 rows
- Validation dataset: 3408 rows
- Test dataset: 3407 rows

We also added an EarlyStopping Callback of patience 10 which automatically stopped training if we do not see any validation accuracy improvement after 10 epochs.

```
Epoch 1/10000
852/852 [==============================] - 11s 10ms/step - loss: 1.1977 - accuracy: 0.4627 - val_loss: 1.0805 - val_accuracy: 0.5161
Epoch 2/10000
852/852 [==============================] - 9s 10ms/step - loss: 1.0930 - accuracy: 0.5132 - val_loss: 1.0625 - val_accuracy: 0.5174
Epoch 3/10000
852/852 [==============================] - 9s 10ms/step - loss: 1.0494 - accuracy: 0.5465 - val_loss: 0.9777 - val_accuracy: 0.5789
Epoch 4/10000
852/852 [==============================] - 9s 10ms/step - loss: 1.0158 - accuracy: 0.5716 - val_loss: 0.9707 - val_accuracy: 0.5864
Epoch 5/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.9896 - accuracy: 0.5856 - val_loss: 1.0243 - val_accuracy: 0.5668
Epoch 6/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.9701 - accuracy: 0.5977 - val_loss: 0.9334 - val_accuracy: 0.6190
Epoch 7/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.9557 - accuracy: 0.6036 - val_loss: 1.0162 - val_accuracy: 0.5668
Epoch 8/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.9384 - accuracy: 0.6118 - val_loss: 0.9276 - val_accuracy: 0.6102
Epoch 9/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.9172 - accuracy: 0.6206 - val_loss: 1.1872 - val_accuracy: 0.5215
Epoch 10/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.9054 - accuracy: 0.6286 - val_loss: 0.9491 - val_accuracy: 0.6086
Epoch 11/10000
852/852 [==============================] - 9s 10ms/step - loss: 0.8890 - accuracy: 0.6372 - val_loss: 0.9704 - val_accuracy: 0.5879
Epoch 12/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.8593 - accuracy: 0.6549 - val_loss: 1.0264 - val_accuracy: 0.5836
Epoch 13/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.8401 - accuracy: 0.6607 - val_loss: 0.9895 - val_accuracy: 0.5970
Epoch 14/10000
852/852 [==============================] - 9s 10ms/step - loss: 0.8155 - accuracy: 0.6744 - val_loss: 0.9790 - val_accuracy: 0.6005
Epoch 15/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.7853 - accuracy: 0.6884 - val_loss: 1.0477 - val_accuracy: 0.5800
Epoch 16/10000
852/852 [==============================] - 8s 10ms/step - loss: 0.7653 - accuracy: 0.6952 - val_loss: 1.0013 - val_accuracy: 0.5967
```

Figure 10: Training Log for the 5-class Classification Model

Here we can see that as Training Stopped at epoch 16, our best model was in epoch 6 which had a validation accuracy of 61.90%

After some trial and error, we found that the best model we could make by altering the network size, had 2.7 million parameters for the five-class classification and 20.4 million parameters for the four-class model

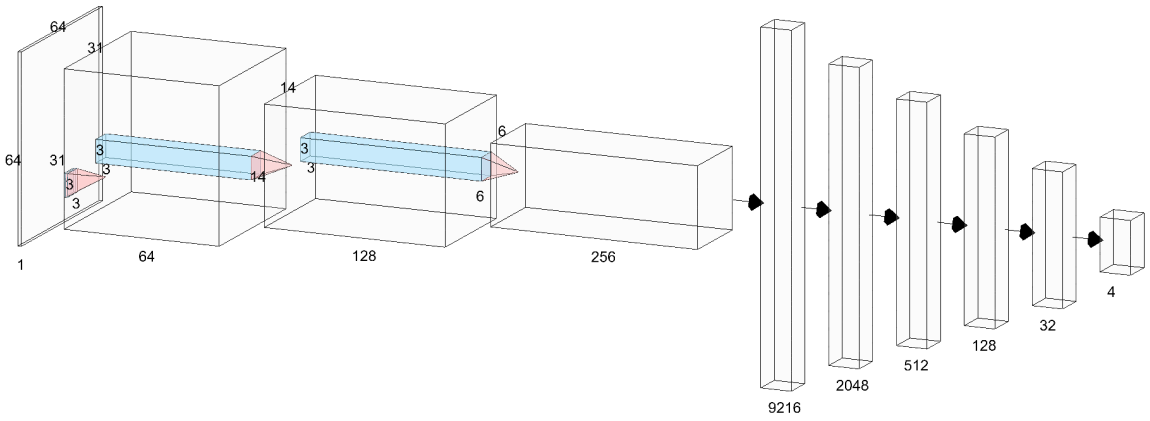


Figure 11: Model Diagram for 4 Class Classifier

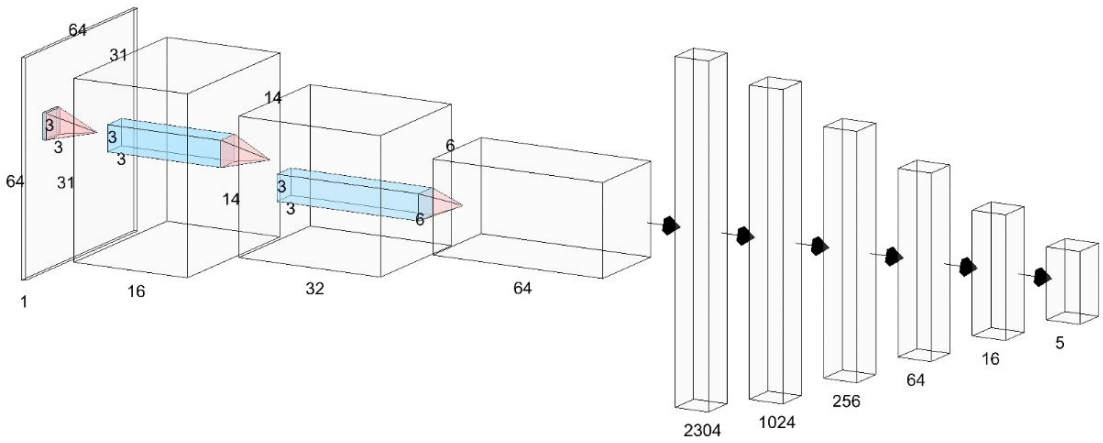


Figure 12: Model Diagram for 5 Class Classifier

We were getting the test accuracy **61.90% for four class classifier and 59.71% for the five-class classifier**

5.2 Model using Transformer

Here we used GPT-neo with 125 million parameters and fine-tuned it with our generated dataset.

Our dataset consisted of 34075 rows that were split for training, testing, and validating in the ratio 80:10:10. The following are the sizes for each split:

- Train dataset: 27260 rows
- Validation dataset: 3408 rows
- Test dataset: 3407 rows

In the beginning, each of our input prompts is converted into tokens using GPT2 tokenizer.

Here we see that the input prompt containing ciphertext and result has their corresponding tokens. Since we have used a tokenizer of fixed size, the unused indexes have higher integer values and have been masked by an attention mask containing 0 at that index.

```
print((final_test_dataset[5]))
print(test_dataset[5])
```

```
Cyphertext: ymccaygimgamkyyiycuycqocyeagmigkycmyyoyy
Algorithm: affine

(tensor([   34,  4464,   372,  5239,    25,   331,    76,   535,   323,    70,
          9600,   321,  2584,    88,  7745,    66,  4669,    66,    80,   420,
          5948,   363,    76,   328,  2584,    66,  1820,    88,   726,    88,
           198,  2348, 42289,    25,  1527,   500,   198, 50256, 50256, 50256,
         50256, 50256, 50256, 50256, 50256, 50256, 50256, 50256, 50256, 50256,
         50256, 50256, 50256, 50256, 50256]), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0]))
```

Figure 13: GPT2 Tokenizer in action

The training and validation data was fit into the model each having a batch size of 32 and was run for 5 epochs.

```
Num examples = 27260
Num Epochs = 5
Instantaneous batch size per device = 32
Total train batch size (w. parallel, distributed & accumulation) = 32
Gradient Accumulation steps = 1
Total optimization steps = 4260
Number of trainable parameters = 125198592
```

Figure 14: Training Parameters for the GPT-Neo Fine Tuning

[4260/4260 37:12, Epoch 5/5]

Step	Training Loss	Validation Loss
500	1.920900	1.839412
1000	1.826400	1.815576
1500	1.794200	1.798747
2000	1.760900	1.792532
2500	1.746500	1.784798
3000	1.721100	1.785910
3500	1.708000	1.788620
4000	1.688200	1.787953

Figure 15: Training and Validation Loss obtained every 500 steps

After fine-tuning, we tested the model on our test dataset containing 3407 rows. The model has shown an **overall accuracy of 83.24%**.

```
print(f"Correct: {correct}, Total: {total}")
print(f"Accuracy: {correct/total}")
```

```
Total test dataset size = 3407

Accuracy 0.8324038743762842

Completed 100.0%

Computed 3407 results
Correct: 2836, Total: 3407
Accuracy: 0.8324038743762842
```

Figure 16: Testing Accuracy for the Fine-Tuned GPT-Neo Model

6 FUTURE SCOPE

We would also like to alter the text-to-matrix conversion for the dataset to make the CNN model perform better as it is not performing up to the standards that we want.

We would like to finetune our models to see if we can increase the accuracy. We are also curious to see if we can use an RNN instead of transformers and record the difference in the accuracy of the two.

A similar analysis can be performed on other models such as:

- OPT
- BERT
- GPTJ
- Bloom

Apart from this, we would like to explore feature detection to create a model and compare this approach with the transformer-based model.

In future work, we would like to extend the number of classes of ciphertext and also increase our dataset to find how our models perform in the new environment. We would like to try a feature-learning approach and also use more transformer models. We would also like to solve the Cipher ID challenge from Mystery Twister C3 consisting of 500 ciphertexts. Based on our results and improvements, we would like to publish our work in a journal to see how our results stand in global competition.

7 CONCLUSION

This report shows the current progress of our work in the detection of ciphertext using Artificial Neural Networks. Here we used two types of Neural Networks to achieve our objective, namely: Convolution Neural Networks and Transformer approach.

With the CNN approach, our model was able to achieve an accuracy of 61.90% for 5 class classifications. We saw a significant improvement with the transformers approach on the same dataset and classes giving an accuracy of 83.24%. This was primarily due to the higher number of parameters of Gpt-neo and the ability to memorize the local importance of tokens due to the self-attention feature.

REFERENCES

[1] "From the game map to the battlefield – using deepmind's advanced alphastar techniques to support military decision-makers." https://www.sto.nato.int/publications/STO%20Meeting%20Proceedings/STO-MP-MSG-184/MP-MSG-184-14.pdf. (Accessed on 12/04/2022).

[2] "Welcome to mysterytwister! — mysterytwister — the cipher contest." https://mysterytwister.org/home/welcome/.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017.

[4] N. Kopal, "Of ciphers and neurons – detecting the type of ciphers using artificial neural networks," pp. 77–86, 05 2020.

[5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[7] E. Leierzopf, V. Mikhalev, N. Kopal, B. Esslinger, H. Lampesberger, and E. Hermann, "Detection of classical cipher types with feature-learning approaches," in *Australasian Conference on Data Mining*, pp. 152–164, Springer, 2021.

[8] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, pp. 1–10, 2022.

[9] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, *et al.*, "Gpt-neox-20b: An open-source autoregressive language model," *arXiv preprint arXiv:2204.06745*, 2022.

[10] "Fine-tune a pretrained model." https://huggingface.co/docs/transformers/training. (Accessed on 12/04/2022).

[11] "Fine-tuning transformers for nlp." https://www.assemblyai.com/blog/fine-tuning-transformers-for-nlp/. (Accessed on 12/04/2022).

[12] "How to generate text: using different decoding methods for language generation with transformers." https://huggingface.co/blog/how-to-generate. (Accessed on 12/04/2022).